

Parnassus: Neural Fast Simulation and Reconstruction for High Energy Physics

Etienne Dreyer^a, Eilam Gross^a, Dmitrii Kobylanskii^a, Vinicius Mikuni^b, Benjamin Nachman^b

^a*Weizmann Institute of Science, Rehovot, Israel*

^b*Lawrence Berkeley National Laboratory, Berkeley, CA, USA*

Abstract

We present the public software release of PARNASSUS, a pure-Python, GPU-compatible framework for neural fast detector simulation and reconstruction in High Energy Physics. Parnassus replaces the computationally expensive GEANT4 simulation and reconstruction chain with conditional flow matching models that map truth-level particles directly to detector-level particle-flow objects. The framework is both process-agnostic and detector-agnostic: users select a detector model—analogue to choosing a detector card in DELPHES—and the same pre-trained model can be applied to arbitrary physics processes without retraining. Unlike C++/ROOT-based tools such as DELPHES, Parnassus runs natively on GPU (CUDA and Apple MPS) and requires no ROOT installation. The current release ships with a CMS detector model trained on 2011 open data; additional detector models will be provided in future releases. We describe the installation, command-line and Python API, configuration system, and demonstrate the framework on Standard Model and BSM processes.

Keywords: fast simulation, detector simulation, particle flow, generative models, flow matching, diffusion models, high energy physics

PROGRAM SUMMARY

Program title: Parnassus

CPC Library link to program files:

(to be added by Technical Editor)

Developer’s repository link: <https://github.com/parnassus-hep/parnassus>

Licensing provisions: MIT

Programming language: Python (≥ 3.12)

Nature of problem:

Full detector simulation and reconstruction based on GEANT4 is computationally expensive, often dominating the computing budget of large-scale High Energy Physics analyses. A fast, accurate surrogate is needed that maps truth-level (generator-level) particles directly to detector-level particle-flow objects, preserving the fidelity of the full chain while being orders of magnitude faster.

Solution method:

Parnassus trains conditional flow matching models [7, 8] on full simulation and reconstruction data. A neural detector model, conditioned on truth-level particle properties, generates detector-level particle-flow objects including kinematics, classification, and track impact parameters. Generation is followed by physics-based post-processing (jet clustering via FAST-JET [17], lepton isolation). Users select a detector model—analogue to a DELPHES card—to target a specific experiment.

Additional comments including restrictions and unusual features:

The framework is implemented entirely in Python with PyTorch and runs natively on CPU, CUDA GPU, and Apple MPS. It has no dependency on ROOT; output to ROOT format is optionally handled via `uproot` [24]. The current release ships with a CMS detector model trained on 2011 open data [6]; additional detector models for other experiments will be added in future releases. The CMS model supports up to 400 truth particles per event; events exceeding this limit retain the 400 highest- p_T particles.

1 Introduction

Parnassus is a **pure-Python, fully GPU-compatible** framework for **fast detector simulation and reconstruction** in High Energy Physics (HEP). It replaces the computationally expensive full GEANT4 [3] simulation and reconstruction chain with neural network-based generative models that learn the mapping from truth-level (generator-level) particles directly to detector-level particle-flow [4, 5] (pflow) objects, bypassing intermediate steps such as hit simulation, digitisation, and track reconstruction. The framework is both **process-agnostic** and **detector-agnostic**: the same architecture supports multiple detector models, and for each detector the pre-trained model can be applied to arbitrary physics processes by simply providing different input truth events.

Unlike traditional fast simulation tools such as DELPHES [25], which are written in C++, require the ROOT framework [19], and run only on CPU, Parnassus is implemented entirely in Python with PyTorch [22] and runs natively on GPU (CUDA and Apple MPS), enabling seamless integration into modern Python-based analysis workflows and significant acceleration on GPU hardware. ROOT I/O is supported via `uproot` [24] for writing output files, but is entirely optional — Parnassus has no dependency on ROOT itself.

Users select a **detector model** — analogous to choosing a detector card in DELPHES — to target a specific experiment. The current release ships with a CMS detector model trained on 2011 open data [6]; additional detector models for other experiments will be added in future releases.

The physics methodology underlying Parnassus has been described in detail in Refs. [1, 2], where we demonstrated that the approach reproduces full CMS simulation and reconstruction across a wide range of Standard Model and BSM processes. In this document we focus on the **public software release** — its installation, usage, and API — to enable the broader HEP community to adopt neural fast simulation and reconstruction in their workflows.

The neural backbone uses conditional flow matching [7, 8] to train continuous-time diffusion models [9, 10] that are sampled via Euler integration at inference time. The models are conditioned on truth-level particle properties, making the generation inherently process-agnostic without requiring retraining for new physics scenarios.

1.1 Design goals

Speed Orders of magnitude faster than full simulation and reconstruction by replacing the detector response with neural diffusion models. Fully GPU-compatible for additional acceleration.

Pure Python

Implemented entirely in Python with no compiled dependencies beyond PyTorch. Unlike C++/ROOT-based tools such as DELPHES [25], Parnassus integrates naturally into modern Python analysis ecosystems. ROOT output is available via `uproot` but is not required.

Fidelity	Each detector model is trained on full simulation and reconstruction data to reproduce realistic detector-level distributions including particle kinematics, vertex positions, impact parameters, and particle identification. Excellent agreement with full simulation has been demonstrated across diverse SM and BSM processes [1, 2].
Flexibility	Accept truth-level input from any source — PYTHIA8 [11] on-the-fly generation, HepMC [12] files from external generators (MADGRAPH5_AMC@NLO [13], SHERPA [14], HERWIG [15], POWHEG [16]), or custom formats. Users select a detector model to target a specific experiment, analogous to choosing a detector card in DELPHES.
Completeness	Full post-processing pipeline including jet clustering (FASTJET [17]) and lepton isolation, with optional output to ROOT format via uproot [24].

1.2 Output quantities

Given a set of truth-level particles for each event, Parnassus generates the quantities listed in Table 1.

Table 1: Output quantities produced by Parnassus for each event.

Output	Description
Particle-flow particles	Full kinematics (p_T, η, ϕ), vertex (v_x, v_y, v_z), class, PDG ID
Impact parameters	d_0, z_0 and their errors for charged particles
Jets	Clustered with configurable algorithms (anti- k_t [18], gen- k_t); substructure (D_2 [20], C)
Lepton isolation	Isolation scores and p_T sums in configurable ΔR cones
Event-level quantities	$H_T, \vec{E}_T^{\text{miss}}$ components, particle multiplicity

2 Architecture overview

Parnassus passes truth-level events through a neural detector model followed by physics-based post-processing. The pipeline is illustrated in Fig. 1.

The neural detector model is a **diffusion model** trained with conditional flow matching [7, 8] and sampled via Euler integration. The model is conditioned on truth-level particle information, making the generation process inherently dependent on the input physics process without being tied to any specific one. Generated output variables include particle-flow kinematics (p_T, η, ϕ), vertex positions, particle classification, and track impact parameters (d_0, z_0).

The number of diffusion steps is configurable at runtime via the `--num_steps` (`-n`) flag, allowing users to trade generation speed for quality.

3 Installation and quick start

3.1 Installation

```
# Clone the repository
git clone https://github.com/parnassus-hep/parnassus.git
cd parnassus

# Install with pip (requires Python 3.12)
```

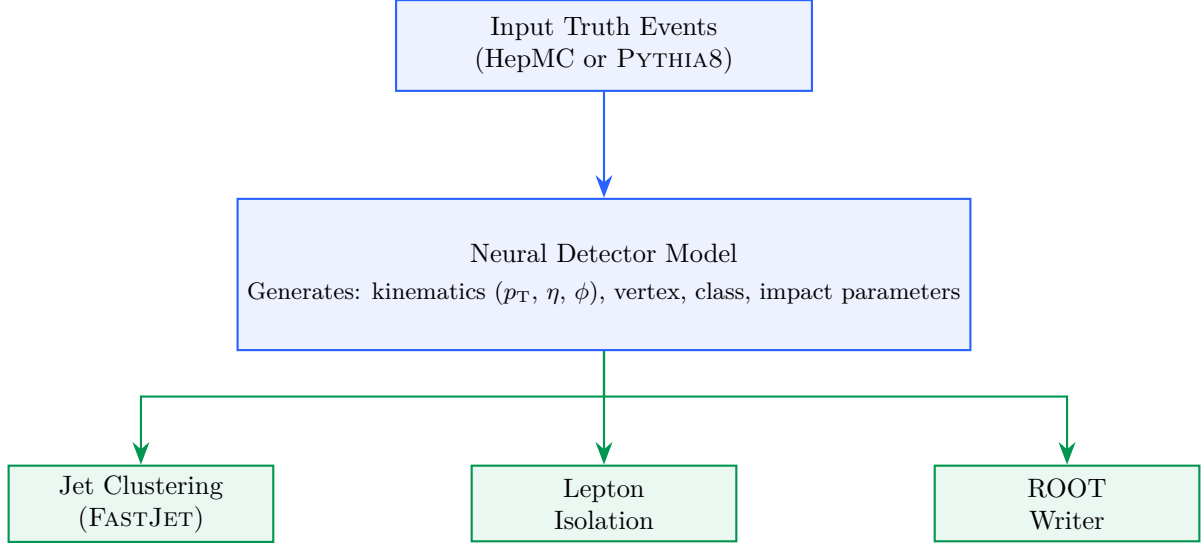


Figure 1: Parnassus generation pipeline. The neural detector model (blue) produces detector-level particle-flow objects from truth-level input. Physics-based post-processing stages (green) compute derived quantities and write the final output.

```

121 pip install .
122
123 # For development
124 pip install -e ".[dev]"
125

```

Listing 1: Installation commands.

3.2 Requirements

- Python ≥ 3.12 , < 3.13
- PyTorch [22] $\geq 2.6.0$
- Pythia8mc [11] 8.316.0
- PyHepMC [12] $\geq 2.14.0$
- FastJet [17] $\geq 3.4.3.1$
- EnergyFlow [23] $\geq 1.4.0$
- Uproot [24] $\geq 5.5.2$ (*optional, for ROOT output*)

3.3 Initialise a configuration file

```

135 parnassus init .
136
137

```

This copies the default `default_config.yaml` to the current directory. The default configuration uses the `cms_2011_flow_v00` detector model with anti- k_t jet clustering and lepton isolation. To target a different detector, change the `generator.name` field to the desired detector model.

3.4 Run with a HepMC input file

```

parnassus run \
  -c default_config.yaml \
  -i events.hepmc \
  -ne 1000 \
  -bs 2000 \
  -o output.root

```

Listing 2: Basic command-line invocation.

The available command-line flags are listed in Table 2.

Table 2: Command-line flags for `parnassus run`.

Flag	Long form	Description
-c	--config	Path to YAML configuration file
-i	--input_path	Input HepMC or Pythia <code>.cmnd</code> file
-o	--output_path	Output ROOT file path
-ne	--num_events	Number of events to process
-bs	--batch_size	Batch size for neural generation
-n	--num_steps	Diffusion sampler steps (default: 50)

3.5 Run with Pythia8 on-the-fly generation

To generate truth-level events on-the-fly with PYTHIA8, create a `.cmnd` steering card and pass it as the input file. Parnassus detects `.cmnd` files automatically:

```

parnassus run \
  -c default_config.yaml \
  -i ttbar.cmnd \
  -ne 5000 \
  -bs 2000 \
  -o ttbar_fastsim.root

```

4 Demonstration: SM and BSM processes

The neural models are **process-agnostic** — they learn the detector response as a function of truth-level particle properties, not the physics process itself. We have previously demonstrated [1, 2] that Parnassus reproduces full simulation and reconstruction across a wide range of SM and BSM processes. Here we show two representative examples using the CMS detector model to illustrate the API in action: a Standard Model process ($H \rightarrow ZZ \rightarrow 4\ell$) and a BSM exotic Higgs decay ($H \rightarrow aa \rightarrow gg\mu\mu$). The same workflow applies to any detector model by changing the `generator.name` in the configuration.

The current CMS detector model supports up to 400 truth particles per event. For events exceeding this limit, Parnassus retains the 400 highest- p_T particles so that no events are discarded.

4.1 Standard Model: $H \rightarrow ZZ \rightarrow 4\ell$

We process 100 events from a HepMC input file produced by PYTHIA8 [11]:

```

176
177 parnassus run \
178   -c default_config.yaml \
179   -i h4l_events.hepmc \
180   -ne 100 \
181   -o h4l_output.root
182

```

183 4.2 BSM: $H \rightarrow aa \rightarrow gg\mu\mu$

184 For the BSM example, we generate exotic Higgs decays on-the-fly with PYTHIA8. The Higgs
 185 decays to a pair of light pseudoscalars a ($m_a = 20$ GeV), each decaying to either gg or $\mu^+\mu^-$,
 186 producing a mixed final state of jets and muon pairs:

```

187
188 ! haa_ggmumu.cmnd -- BSM exotic Higgs decay
189 Beams:eCM = 13000.
190 Beams:idA = 2212
191 Beams:idB = 2212
192
193 ! BSM Higgs production via gluon fusion
194 Higgs:useBSM = on
195 HiggsBSM:gg2H2 = on
196 35:m0 = 125.0 ! Higgs mass
197
198 ! SM-like couplings for production
199 HiggsH2:coup2u = 1.0
200 HiggsH2:coup2d = 1.0
201 HiggsH2:coup2A3A3 = 1.0 ! H -> aa coupling
202
203 ! Force H -> a a only
204 35:onMode = off
205 35:onIfAll = 36 36
206
207 ! Light pseudoscalar a properties (PDG ID 36)
208 36:m0 = 20.0 ! mass = 20 GeV
209 HiggsA3:coup2d = 1.0
210 HiggsA3:coup2u = 1.0
211 HiggsA3:coup2l = 1.0
212
213 ! Force a -> gg or mumu only
214 36:onMode = off
215 36:onIfAny = 21 13
216

```

Listing 3: Pythia8 steering card for BSM $H \rightarrow aa \rightarrow gg\mu\mu$.

```

217
218 parnassus run \
219   -c default_config.yaml \
220   -i haa_ggmumu.cmnd \
221   -ne 1000 \
222   -o haa_ggmumu_output.root
223

```

224 This demonstrates running Parnassus on a BSM signal process not present in the training data.
 225 The mixed $gg\mu\mu$ final state probes the model’s ability to simultaneously generate hadronic jets
 226 and isolated muons within the same event.

4.3 Kinematic distributions

Fig. 2 compares the particle-flow kinematic distributions for both processes. The p_T spectra follow the expected steeply falling shape, the η distributions reflect the detector acceptance, and the multiplicity distributions reflect the different underlying event structures. The BSM $H \rightarrow aa$ process shows higher multiplicity due to the hadronic activity from the $a \rightarrow gg$ decays.

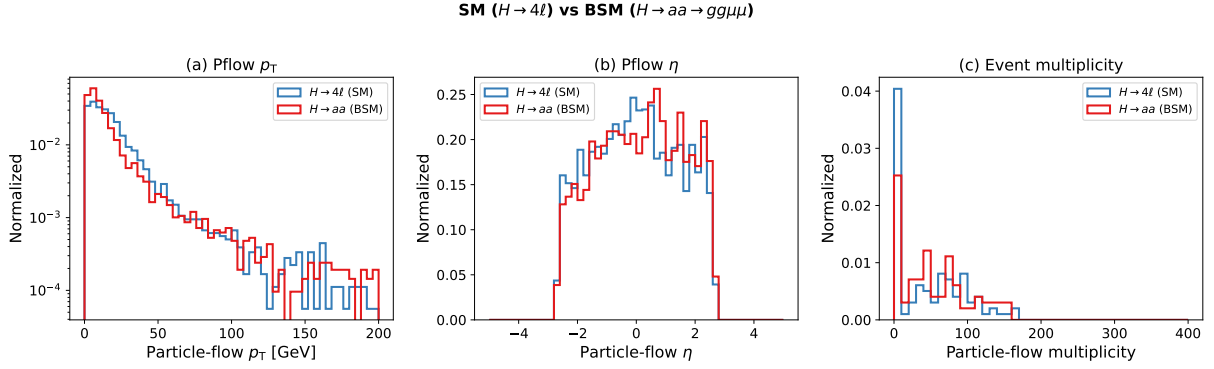


Figure 2: Particle-flow kinematic distributions comparing SM ($H \rightarrow 4\ell$, 99 events) and BSM ($H \rightarrow aa \rightarrow gg\mu\mu$, 99 events) processes. (a) p_T in log scale, (b) pseudorapidity η , (c) particle-flow multiplicity per event. All distributions are normalised to unit area.

4.4 Event display

Fig. 3 shows a representative $H \rightarrow 4\ell$ event in the η - ϕ plane, illustrating how Parnassus transforms truth-level particles into detector-level pflow objects.

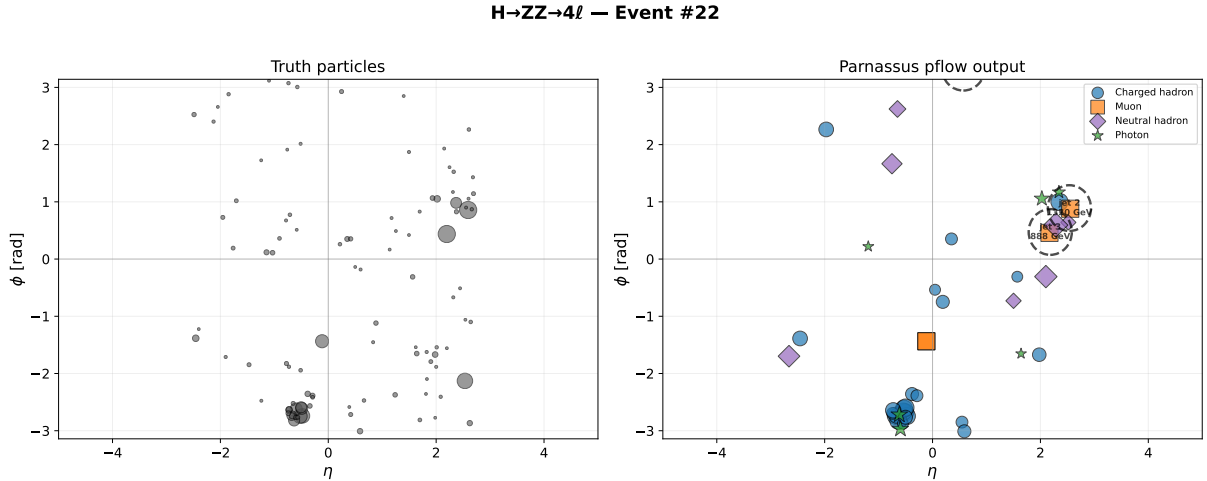


Figure 3: Event display for $H \rightarrow ZZ \rightarrow 4\ell$: truth particles (left) and Parnassus pflow output (right) in the η - ϕ plane. Marker size scales with p_T ; dashed circles indicate anti- k_t jets ($R = 0.5$). Particle classes: charged hadrons (blue circles), electrons (red triangles), muons (orange squares), neutral hadrons (purple diamonds), photons (green stars).

4.5 Aggregate statistics

Table 3 compares the aggregate statistics for both processes. The BSM process shows higher truth-particle multiplicity (163 ± 70) due to the $a \rightarrow gg$ hadronic decays, while the pflow output multiplicity is comparable across both processes.

Table 3: Aggregate statistics for SM and BSM processes (mean $\pm$ std).

Quantity	$H \rightarrow 4\ell$ (99 evt)	$H \rightarrow aa$ (99 evt)
Truth particles/event	97.2 ± 46.0	163.4 ± 69.9
Pflow particles/event	46.3 ± 47.2	53.6 ± 46.4

4.6 Particle class distribution

Fig. 4 compares the particle class composition between the two processes. Both show similar class fractions dominated by charged hadrons ($\sim 60\%$), consistent with the process-agnostic nature of the model.

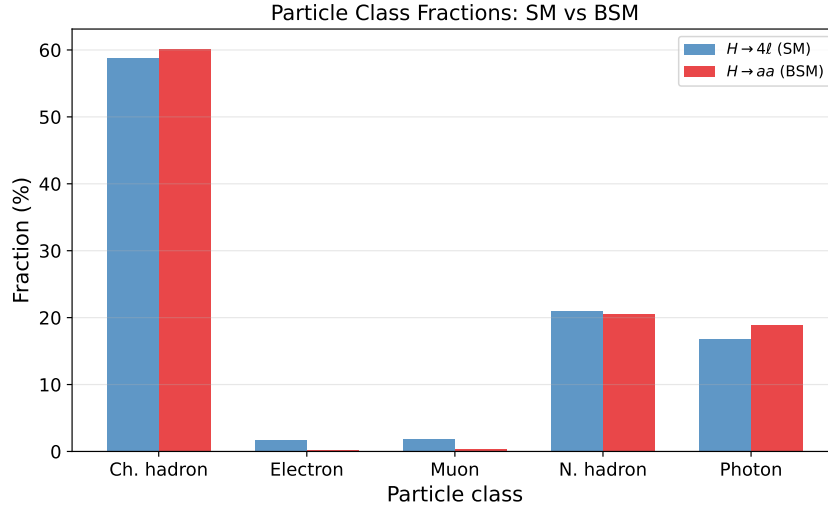


Figure 4: Particle class fractions for SM ($H \rightarrow 4\ell$) and BSM ($H \rightarrow aa \rightarrow gg\mu\mu$) processes. The model produces consistent class compositions regardless of the input physics process.

5 Diffusion steps convergence

The number of diffusion steps N controls the trade-off between generation quality and speed. Fig. 5 shows the convergence of particle-flow kinematic distributions as a function of N for the $H \rightarrow 4\ell$ sample.

Fig. 6 shows that particle class fractions are also stable across step counts.

Table 4 summarises the generation time as a function of diffusion steps on CPU.

Table 4: Generation time vs. diffusion steps (100 events, CPU).

Steps	Wall time
10	$\sim 1:30$
25	$\sim 1:30$
50	$\sim 2:10$
100	$\sim 5:20$

The default of 50 steps provides a good balance of quality and speed. For quick validation runs, 25 steps yields nearly identical distributions at lower cost.

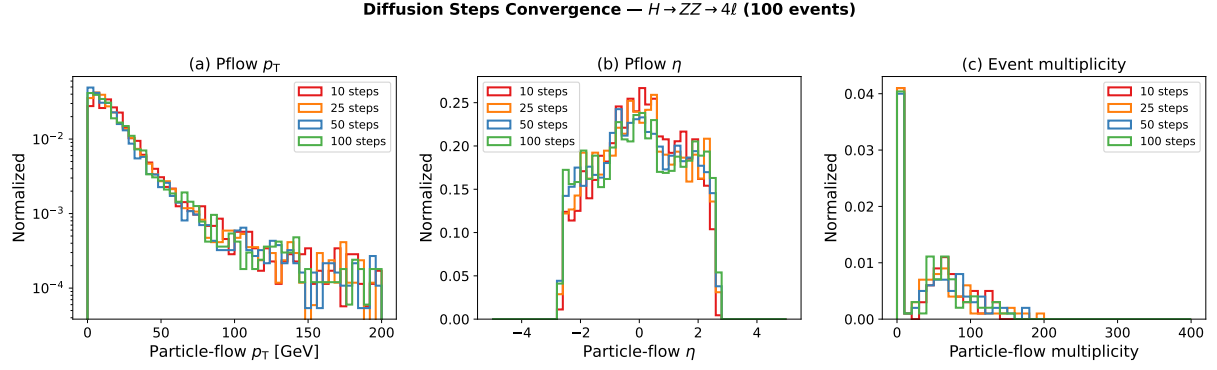


Figure 5: Diffusion steps convergence for $H \rightarrow ZZ \rightarrow 4\ell$ (100 events). (a) Pflow p_T , (b) pflow η , (c) event multiplicity. Distributions are stable from $N = 25$ onward.

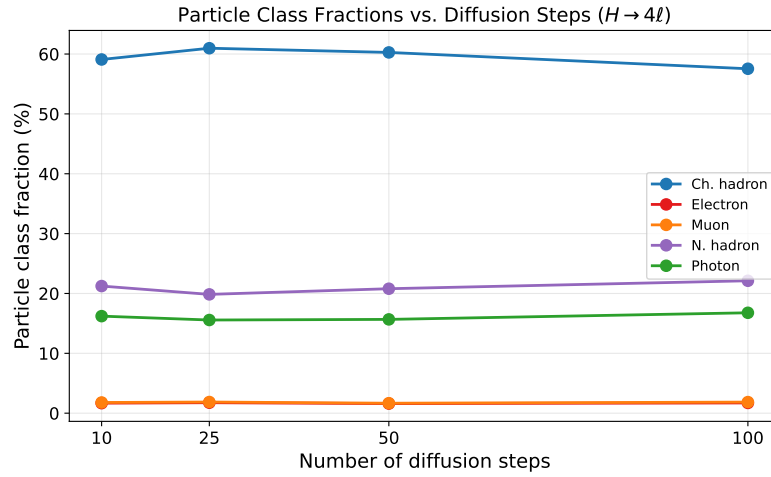


Figure 6: Particle class fractions vs. number of diffusion steps. All classes converge by $N = 25$.

6 Python API

For programmatic usage, Parnassus provides a clean Python API:

```
from pathlib import Path
from parnassus.configs import Config
from parnassus.pipelines import (
    GenerationPipeline,
    JetClusteringPipeline,
    IsolationPipeline,
)
from parnassus.configs.pipeline import (
    JetClusteringConfig,
    IsolationConfig,
)
from parnassus.writers import RootWriter

# Load configuration
config = Config.from_yaml("my_config.yaml")

# Override settings programmatically
config.dataset_config.file_path = (
    Path("my_events.hepmc").absolute()
)
config.dataset_config.num_events = 5000

# Run generation
pipeline = GenerationPipeline(config)
gen_events, accessors_dict = pipeline.run()

# Post-processing
for pipeline_config in config.pipeline_configs:
    if isinstance(pipeline_config, JetClusteringConfig):
        jet_pipeline = JetClusteringPipeline(pipeline_config)
        jet_pipeline.process(gen_events)
    elif isinstance(pipeline_config, IsolationConfig):
        iso_pipeline = IsolationPipeline(pipeline_config)
        iso_pipeline.process(gen_events)

# Inspect generated data
for event in gen_events[:5]:
    print(f"Event {event.event_number}:")
    n_truth = len(event.truth_particles.pt)
    n_pflow = len(event.pflow_particles.pt)
    print(f"  Truth particles: {n_truth}")
    print(f"  Pflow particles: {n_pflow}")
    print(f"  HT = {event.ht:.1f} GeV")

# Write to ROOT
writer = RootWriter(config.writer_config)
writer.write(gen_events)
```

Listing 4: Programmatic usage of the generation pipeline.

6.1 Reading output files

```
import uproot
```

```

305 import numpy as np
306
307 f = uproot.open("output.root")
308 tree = f["Parnassus"]
309
310 # Access pflow jet pT
311 jet_pt = tree["PflowJetsAntiKt05.pt"].array()
312
313 # Access electron isolation
314 e_iso = tree["Electrons.iso_var"].array()
315
316 # Access impact parameters
317 d0 = tree["Pflow.d0"].array()
318 z0 = tree["Pflow.z0"].array()

```

Listing 5: Reading the output ROOT file with uproot.

7 Configuration reference

```

321 # -- Dataset -----
322 dataset:
323     file_path: ""           # Path to .hepmc or .cmnd file
324     num_events: 1000        # Number of events to process
325     entry_start: 0          # Starting event index (HepMC)
326
327 # -- Generator -----
328 generator:
329     type: "neural"          # "neural" or "parametric"
330     name: "cms_2011_flow_v00" # Detector model (like a Delphes card)
331     num_steps: 50           # Diffusion steps (neural only)
332     batch_size: 2000        # Events per batch
333     device: "cpu"           # "cpu", "cuda", or "mps"
334
335 # -- Post-processing Pipelines -----
336 pipelines:
337     TruthJetsAntiKt05:
338         type: "cluster"
339         collection: truth
340         dr: 0.5
341         algorithm: antikt
342         pt_min: 10
343         nconst_min: 2
344     PflowJetsAntiKt05:
345         type: "cluster"
346         collection: pflow
347         dr: 0.5
348         algorithm: antikt
349         pt_min: 10
350         nconst_min: 2
351     ElectronIsolation:
352         type: "isolation"
353         collection: "electrons"
354         dr: 0.4
355     MuonIsolation:
356         type: "isolation"
357         collection: "muons"
358

```

```

359     dr: 0.4
360
361 # -- Output -----
362 output:
363     file_path: ""           # Output ROOT file path
364     format: default
365

```

Listing 6: Annotated default configuration.

8 Conclusion

We have presented the public release of Parnassus, a pure-Python, GPU-compatible framework for neural fast simulation and reconstruction in High Energy Physics. Unlike traditional C++/ROOT-based tools such as DELPHES [25], Parnassus runs entirely in Python with PyTorch, supports GPU acceleration (CUDA and Apple MPS), and requires no ROOT installation — ROOT output is optionally handled via `uproot`. The software provides a simple command-line and Python API for generating detector-level particle-flow objects from truth-level input, with full post-processing including jet clustering and lepton isolation.

The framework is both process-agnostic and detector-agnostic. Users select a detector model — analogous to choosing a detector card in DELPHES — to target a specific experiment. The current release ships with a CMS detector model validated against full CMS simulation across diverse SM and BSM processes [1, 2]; additional detector models will be provided in future releases.

The source code is publicly available at <https://github.com/parnassus-hep/parnassus> under the MIT license.

References

- [1] E. Dreyer, E. Gross, D. Kobylanskii, V. Mikuni, B. Nachman, and N. Soybelman, Parnassus: An Automated Approach to Accurate, Precise, and Fast Detector Simulation and Reconstruction, [arXiv:2406.01620](https://arxiv.org/abs/2406.01620) (2024).
- [2] E. Dreyer, E. Gross, D. Kobylanskii, V. Mikuni, and B. Nachman, Conditional Deep Generative Models for Simultaneous Simulation and Reconstruction of Entire Events, [arXiv:2503.19981](https://arxiv.org/abs/2503.19981) (2025).
- [3] S. Agostinelli et al. (GEANT4 Collaboration), Geant4 — a simulation toolkit, Nucl. Instrum. Meth. A **506** (2003) 250.
- [4] CMS Collaboration, Particle-flow reconstruction and global event description with the CMS detector, JINST **12** (2017) P10003, [arXiv:1706.04965](https://arxiv.org/abs/1706.04965).
- [5] ATLAS Collaboration, Jet reconstruction and performance using particle flow with the ATLAS Detector, Eur. Phys. J. C **77** (2017) 466, [arXiv:1703.10485](https://arxiv.org/abs/1703.10485).
- [6] CMS Collaboration, CMS Open Data, <https://opendata.cern.ch/search?experiment=CMS>.
- [7] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, Flow Matching for Generative Modeling, ICLR (2023), [arXiv:2210.02747](https://arxiv.org/abs/2210.02747).
- [8] X. Liu, C. Gong, Q. Liu, Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, ICLR (2023), [arXiv:2209.03003](https://arxiv.org/abs/2209.03003).

- [9] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, B. Poole, Score-Based Generative Modeling through Stochastic Differential Equations, ICLR (2021), [arXiv:2011.13456](#).
- [10] J. Ho, A. Jain, P. Abbeel, Denoising Diffusion Probabilistic Models, NeurIPS (2020), [arXiv:2006.11239](#).
- [11] T. Sjöstrand et al., An introduction to PYTHIA 8.2, Comput. Phys. Commun. **191** (2015) 159, [arXiv:1410.3012](#).
- [12] A. Buckley et al., The HepMC3 event record library for Monte Carlo event generators, Comput. Phys. Commun. **260** (2021) 107310, [arXiv:1912.08005](#).
- [13] J. Alwall et al., The automated computation of tree-level and next-to-leading order differential cross sections, and their matching to parton shower simulations, JHEP **07** (2014) 079, [arXiv:1405.0301](#).
- [14] E. Bothmann et al. (SHERPA Collaboration), Event Generation with Sherpa 2.2, SciPost Phys. **7** (2019) 034, [arXiv:1905.09127](#).
- [15] J. Bellm et al., Herwig 7.0/Herwig++ 3.0 release note, Eur. Phys. J. C **76** (2016) 196, [arXiv:1512.01178](#).
- [16] S. Alioli, P. Nason, C. Oleari, E. Re, A general framework for implementing NLO calculations in shower Monte Carlo programs: the POWHEG BOX, JHEP **06** (2010) 043, [arXiv:1002.2581](#).
- [17] M. Cacciari, G. P. Salam, G. Soyez, FastJet User Manual, Eur. Phys. J. C **72** (2012) 1896, [arXiv:1111.6097](#).
- [18] M. Cacciari, G. P. Salam, G. Soyez, The anti- k_t jet clustering algorithm, JHEP **04** (2008) 063, [arXiv:0802.1189](#).
- [19] R. Brun and F. Rademakers, ROOT — An object oriented data analysis framework, Nucl. Instrum. Meth. A **389** (1997) 81.
- [20] A. J. Larkoski, I. Moult, D. Neill, Power Counting to Better Jet Observables, JHEP **12** (2014) 009, [arXiv:1409.6298](#).
- [21] A. J. Larkoski, G. P. Salam, J. Thaler, Energy Correlation Functions for Jet Substructure, JHEP **06** (2013) 108, [arXiv:1305.0007](#).
- [22] A. Paszke et al., PyTorch: An Imperative Style, High-Performance Deep Learning Library, NeurIPS (2019), [arXiv:1912.01703](#).
- [23] P. T. Komiske, E. M. Metodiev, J. Thaler, EnergyFlow: A Python framework for jet and event analysis, (2019).
- [24] J. Pivarski et al., Uproot: ROOT I/O in pure Python and NumPy, <https://github.com/scikit-hep/uproot5>.
- [25] J. de Favereau et al. (DELPHES 3 Collaboration), DELPHES 3: A modular framework for fast simulation of a generic collider experiment, JHEP **02** (2014) 057, [arXiv:1307.6346](#).
- [26] A. Butter et al., Machine Learning and LHC Event Generation, SciPost Phys. **14** (2023) 079, [arXiv:2203.07460](#).
- [27] C. Krause et al., CaloChallenge: Community comparison of fast calorimeter simulation, (2024), [arXiv:2410.21611](#).